Ready, set, publish!

Write your first  python™ Package



PyCon France 2026 - Biarritz - Linda Kolb

Linda

- afraid of snakes, but Python is ok 🐍
- Python Developer > 7 years
- Lead Data Scientist at SupplyBrain
- Organiser of the DevOpsDays Graz




dev
ops
days



Hands-on workshops



Awesome speakers



devopsdays GRAZ

DevOpsDays Graz 2027? Check our website at Christmas!

https://www.devopsdays.at/
https://www.devopsdays.org/



Vibe Check

Ever copied that utils.py from one project to another?

ERROR: File path does not exist…
ERROR: Module “numpy” does not exist…

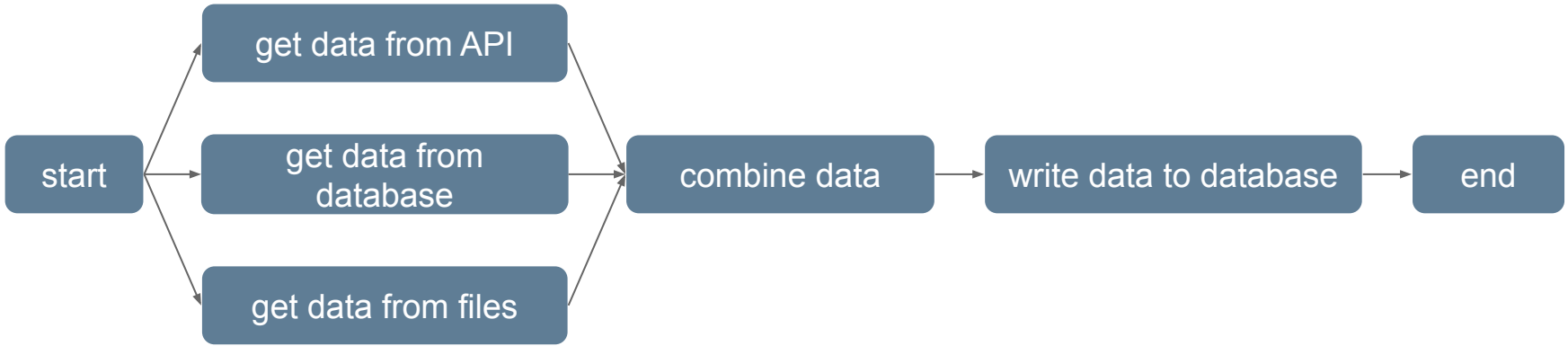


ETL Pipeline

ETL: extract - transform - load

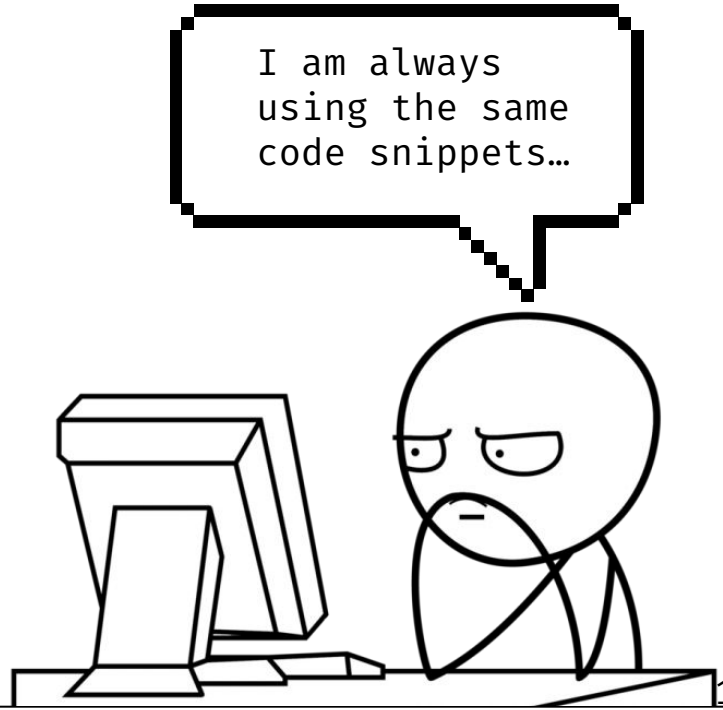


ETL Pipeline



ETL Pipeline



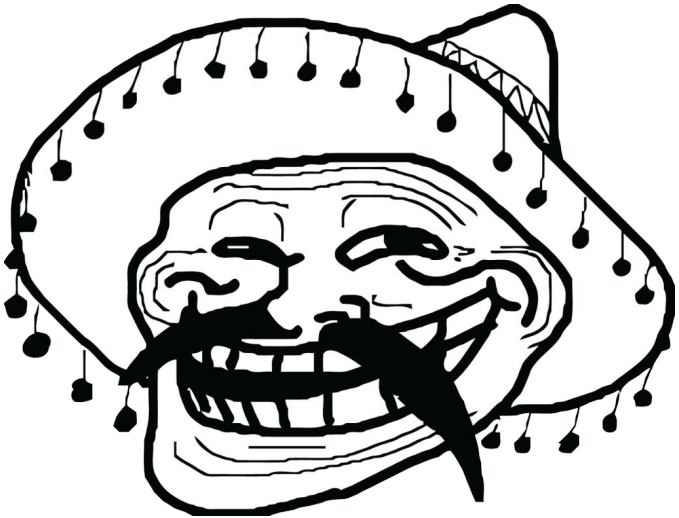


ETL Pipeline

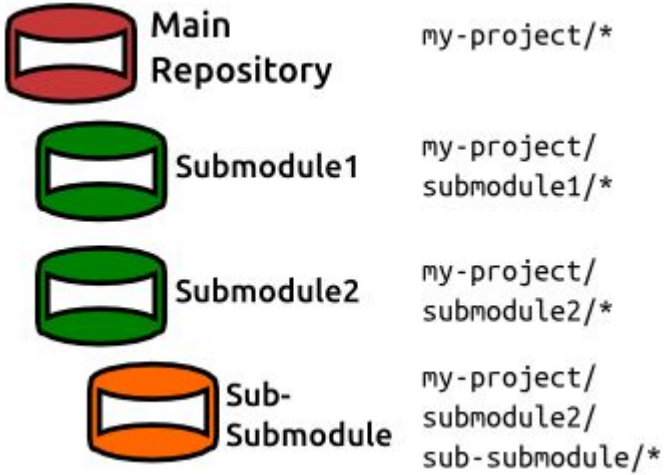




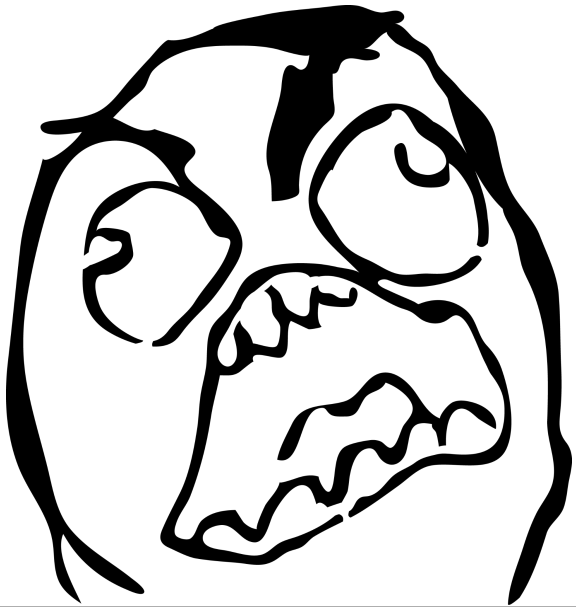
How about git submodules?



A Git submodule is a way to include another Git repo inside your project.



But what about the Python dependencies??



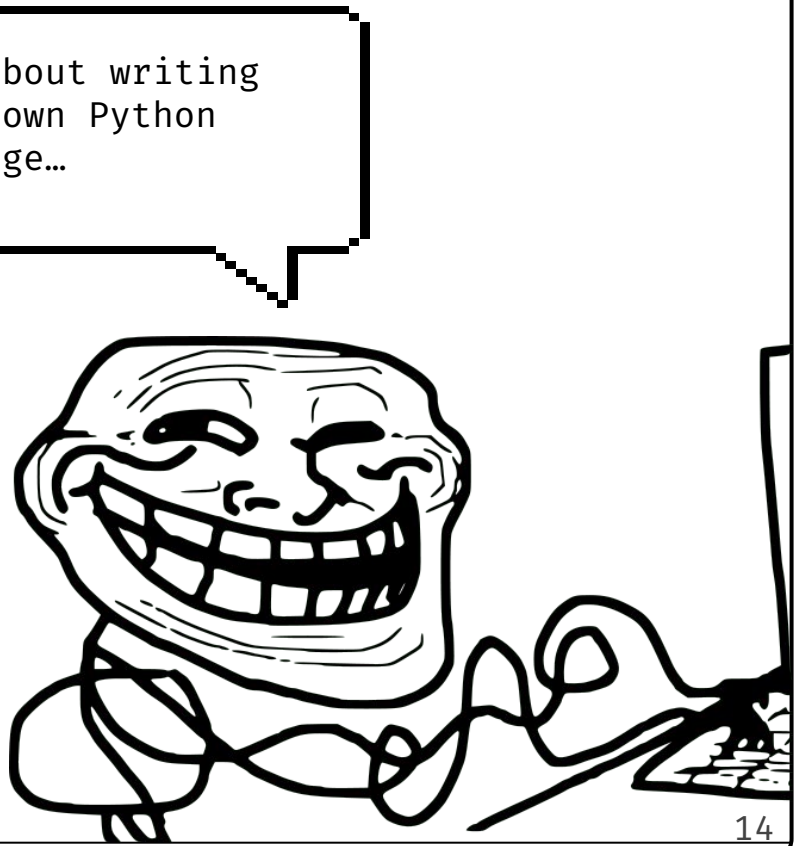
It is possible to add a `requirements.txt/pyproject.toml file` with the submodule, but there is a much cleaner way…



Advantages of a Python package:

- Designed for reuse
- Versioning
- Clean installation
- Dependency resolvement
- Easy collaboration

And the most important thing: You can release your own tiny product to the world!





01 Python Package Structure

What is a Python package?

02 Python Package Dependency Management

uv vs. poetry vs. pip install

03 Build & Publish

Build and publish the package to a Python package registry

04 Package Integration

Install and use the package in another Python project

01
Python Package Structure

What is a Python package?

Python Packages



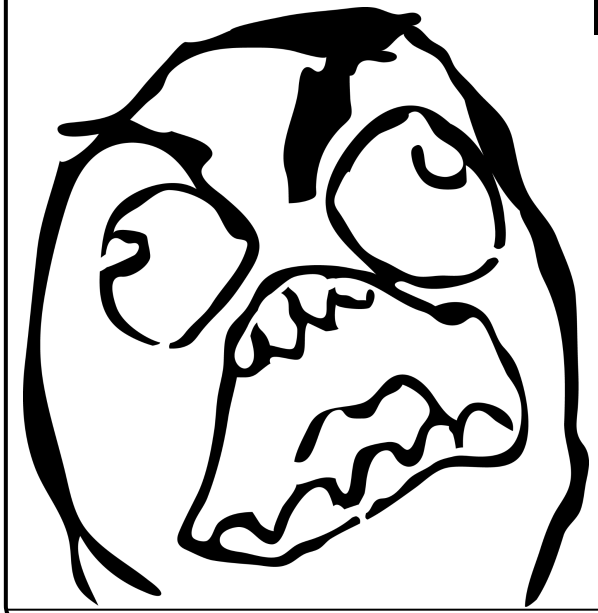
What is a Python Package?

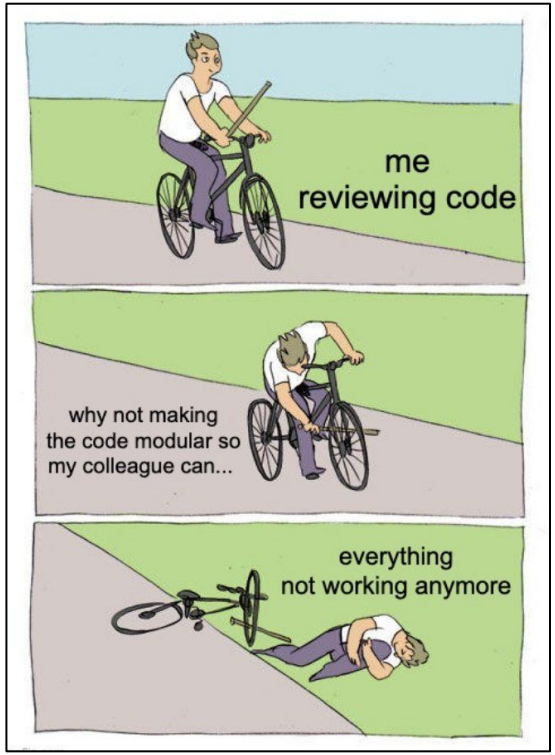
- A collection of Python code you can reuse
- Organised in modules (files) and folders
- Can be imported into other projects

Why?

- Reuse code across projects
- Avoid copy-paste
- Share code with others
- Manage dependencies & versions

But what about
Python modules??





Module vs. Package vs. Library

Module

reusable, importable code file

`column_statistics.py`



Package

consists of related modules

`data_analytics/`

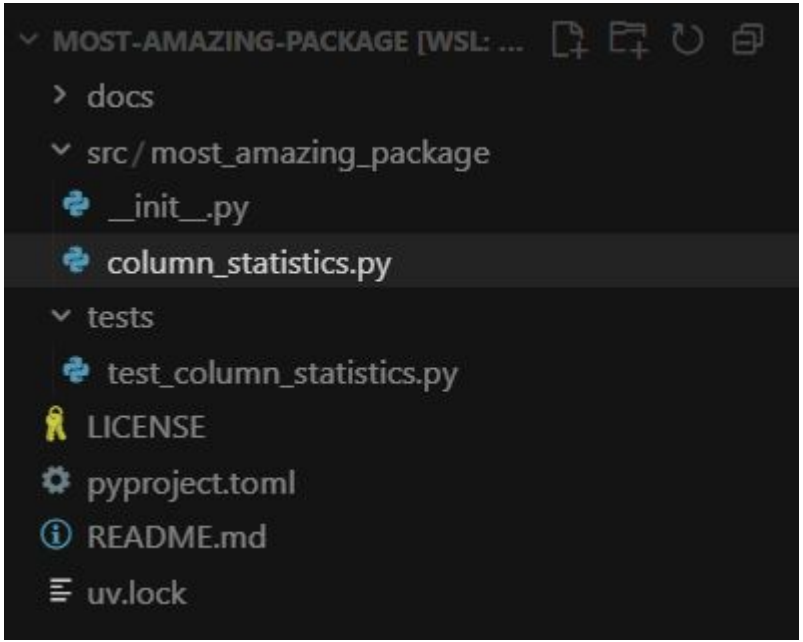


Library

not properly defined

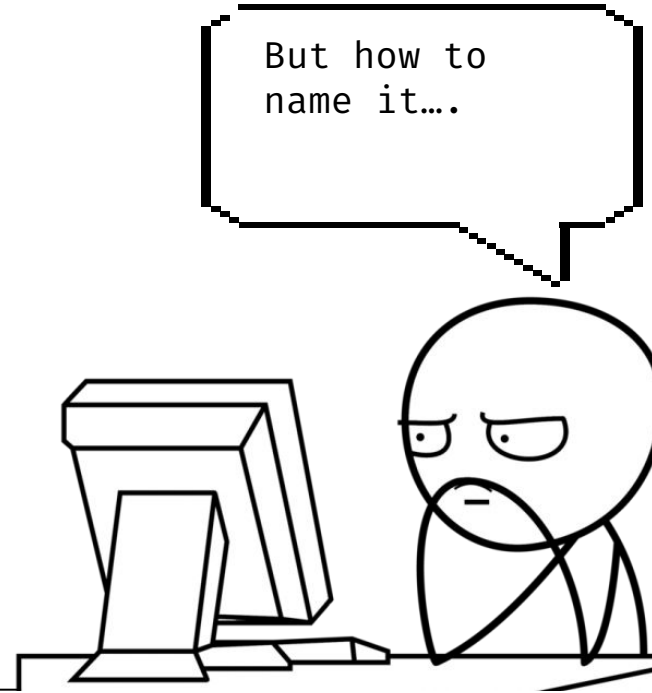


Typical Structure





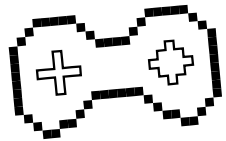

TMP
TEMP_
TEMPORARY
THIS_VARIABLE_WILL_ONLY_
BE_USED_FOR_
A_SHORT_PERIOD_OF_TIME
imgflip.com


But how to
name it….

PEP 8 - Naming Convention

PEP: Python Enhancement Proposal

- Use lowercase letters.
- Hyphens are common for distribution/package names on PyPI, while Python import names typically use underscores, e.g. most_amazing_package
- Use simple and short names
- Do not use abbreviations
- Make sure that the project or package names do not conflict with existing Python standard packages, e.g. pathlib



Docs: https://peps.python.org/pep-0008/

02

Python Package Dependency Management

uv vs. poetry vs. pip install

pip install vs. poetry vs. uv

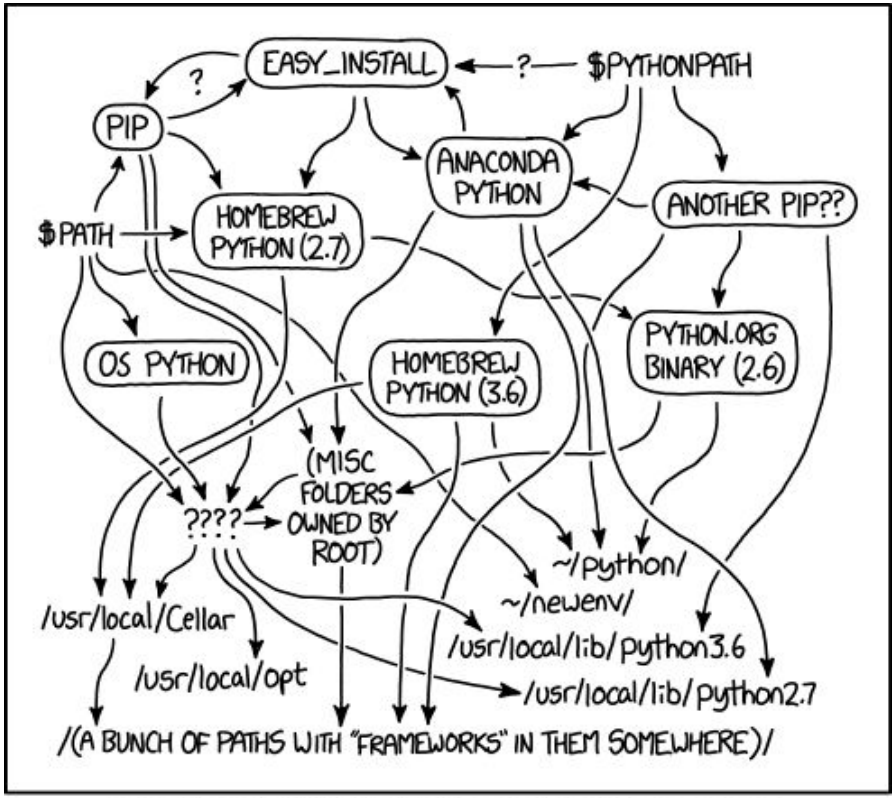
	pip install	**poetry**	**uv**
Initial release	2008	2018	2024
Project configuration and lockfile	command line, requirements.txt (manual pip freeze)	pyproject.toml poetry.lock	pyproject.toml uv.lock
Virtual Environment Management	External (venv, virtualenv)	integrated	integrated
Performance	moderate	moderate	Very Fast (10-100x pip)
Build and publish	Setuptools + twine	Integrated (poetry build/publish)	Integrated (uv build/publish)

Source: https://dimasyotama.medium.com/navigating-the-python-packaging-landscape-pip-vs-poetry-vs-uv-a-developers-guide-49a9c93caf9c

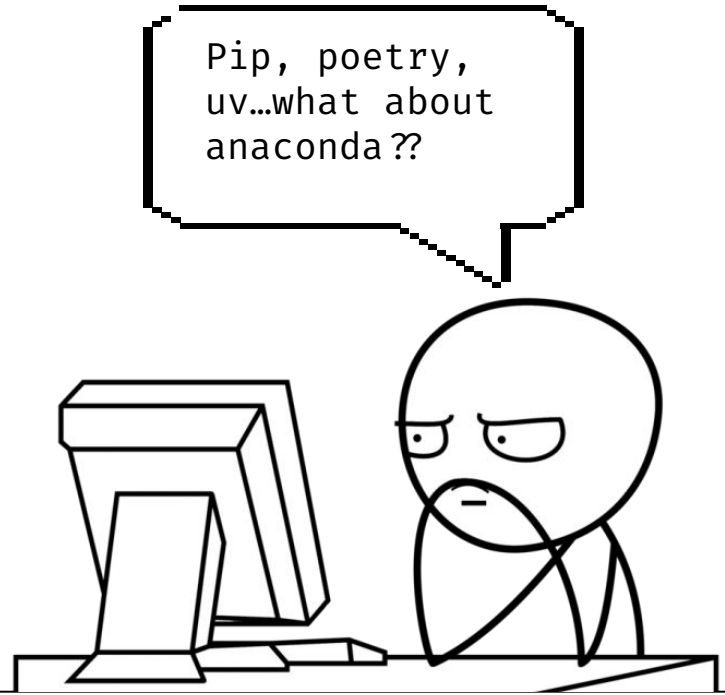


```
sudo add-apt-repository ppa:deadsnakes/ppa
sudo apt update
sudo apt install python3.12
python3.12 -m venv .venv
source .venv/bin/activate
pip-compile requirements.in -o requirements.txt
pip install -r requirements.txt
```

uv sync



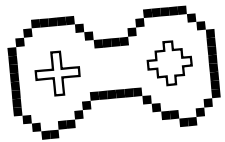
MY PYTHON ENVIRONMENT HAS BECOME SO DEGRADED THAT MY LAPTOP HAS BEEN DECLARED A SUPERFUND SITE.



Conda (Anaconda, Miniconda)

Environment and system package manager (Python + non-Python dependencies)

- Packages come primarily from conda repositories
- Anaconda
  - Proprietary license
  - Will already include many pre-installed packages (data science focus)
- Miniconda
  - Free to use
  - Minimal installer



uv Cheat Sheet

- `uv python install 3.13`
- `uv self update 0.13.0`
- `uv init`
- `uv venv`
- `uv sync`
- `uv add requests`
- `uv add pytest --group dev`
- `uv remove requests`

- `uv run main.py`
- `uv run pytest`
- `uv lock`
- `uv tree`
- `uv build`
- `uv publish`
- `uv sync --upgrade`
- `uvx apache-airflow standalone`

Docs: https://docs.astral.sh/uv/getting-started/features/

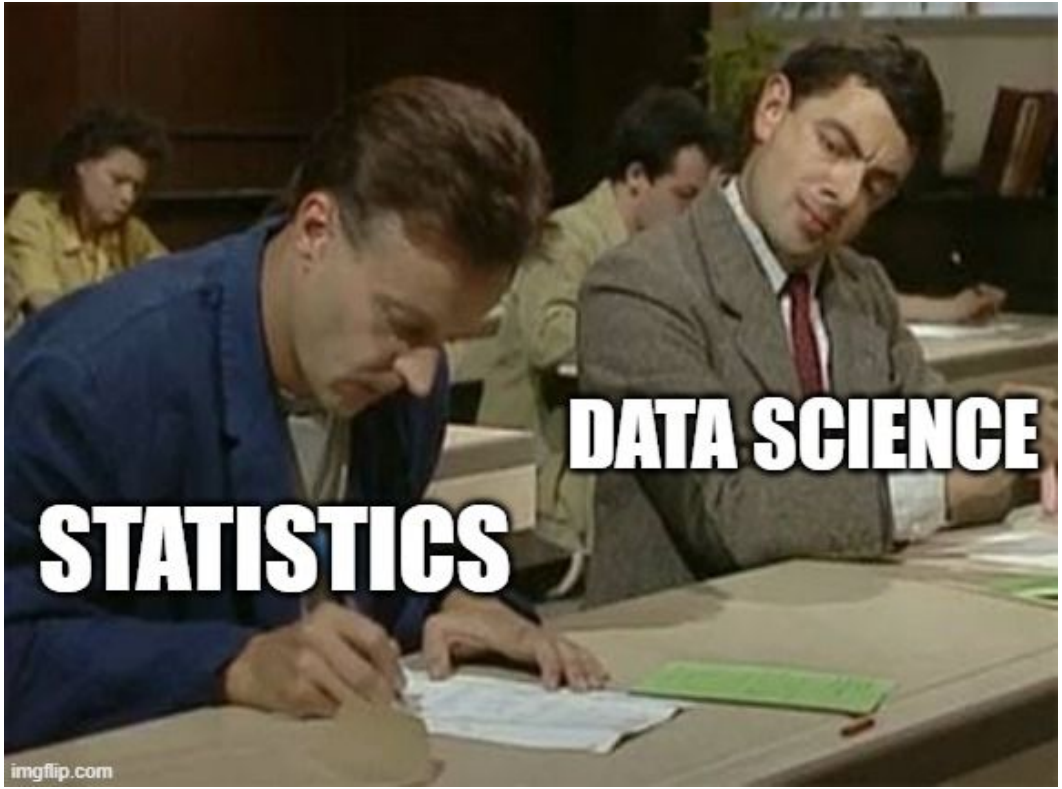
Install uv 0.13.0

Linux + Mac	Windows
curl -LsSf https://astral.sh/uv/install.sh | sh Specific version: curl -LsSf https://astral.sh/uv/0.13.0/install.sh | sh	powershell -ExecutionPolicy ByPass -c "irm https://astral.sh/uv/install.ps1 | iex" Specific version: powershell -ExecutionPolicy ByPass -c "irm https://astral.sh/uv/0.13.0/install.ps1 | iex"

Verify with: `uv --version`

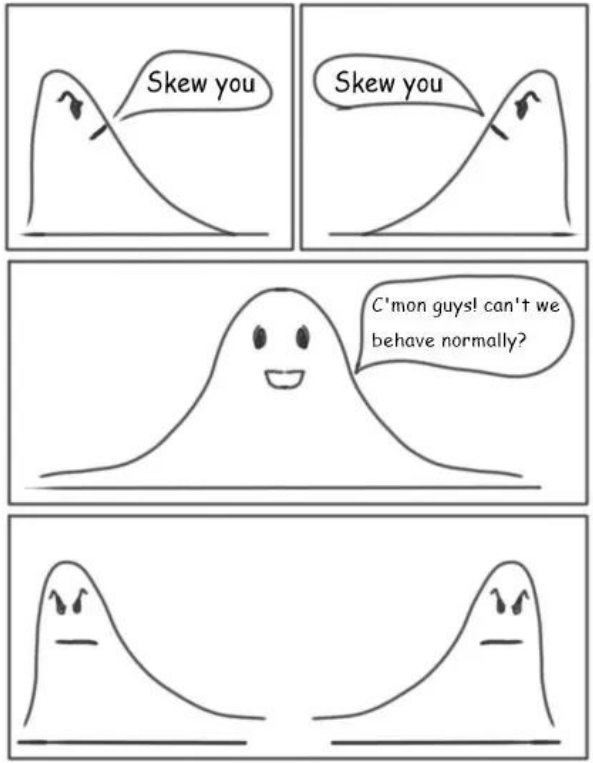
Docs: https://docs.astral.sh/uv/getting-started/installation/#standalone-installer

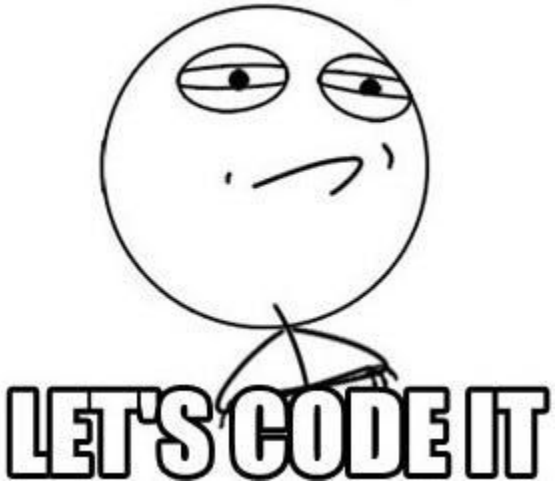
Our Workshop Python Package



Our Workshop Python Package

- Reads in some data
- Calculates metrics based on the data (min, max…)
- Reference repo: https://github.com/kolbl/data-analytics




LET'S CODE IT

Pick your Poison



Visual Studio Code



Cursor IDE

An AI-powered code editor

Create repo folder

1. Create a folder called “data-analytics”
2. Press F1, enter “Install Extensions” and install the Python extension

Create package structure

1. Install Python 3.13
   `uv python install 3.13`

2. Create package structure:
   `uv init --name data-analytics --package --python 3.13`

3. Create virtual environment:
   `uv venv`

4. Select Python interpreter in your IDE

The .venv Folder

A `virtual environment` for your project

- Separate from your global Python
- Keeps dependencies isolated per project
- What's inside?
  - `bin/` or `scripts/` → executables
  - `lib/` → installed packages
  - Python runtime



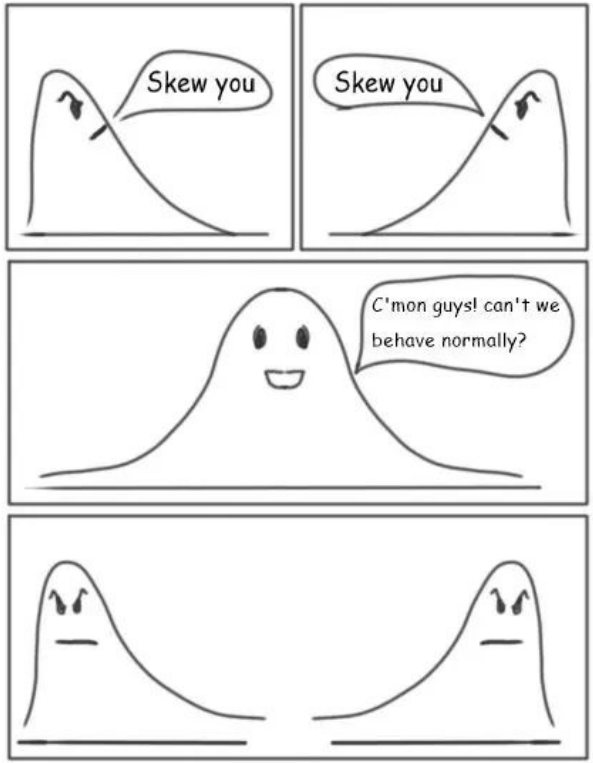
Set up Dependencies

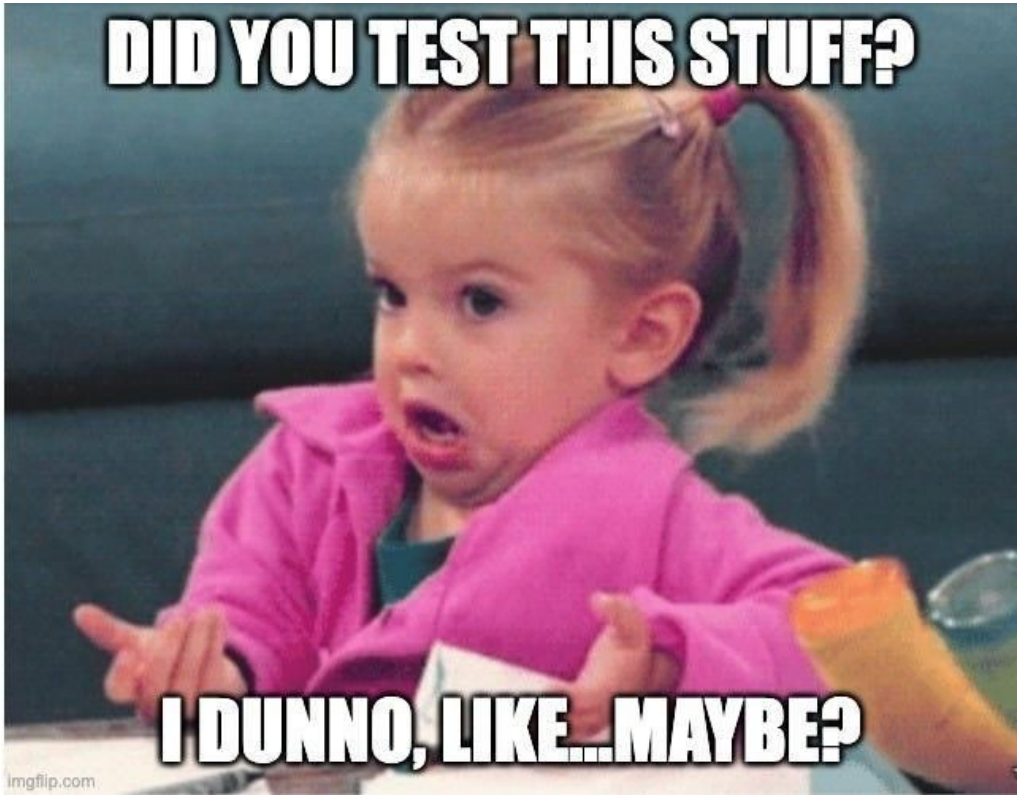
1. Add pandas as dependency:
   `uv add pandas`

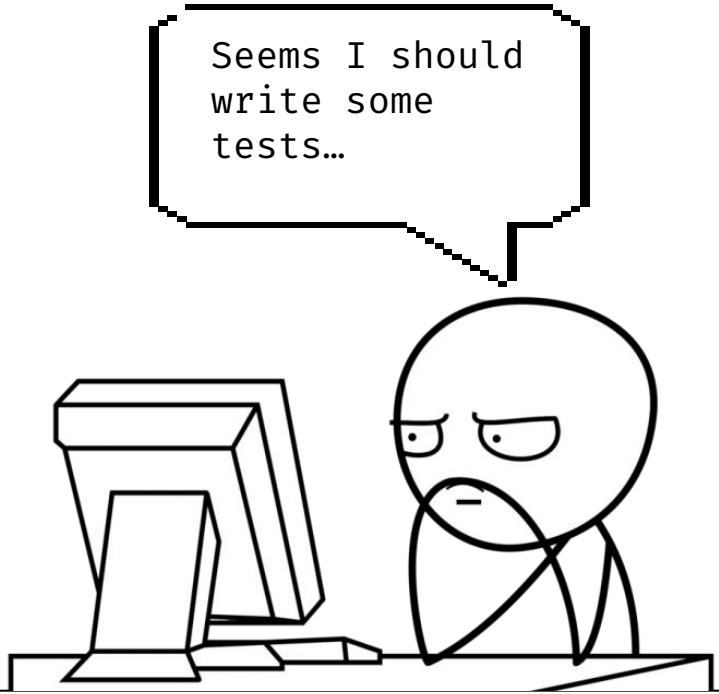
2. Check dependencies:
   `uv tree`

Our Workshop Python Package

- Reads in some data
- Calculates metrics based on the data (min, max…)




DID YOU TEST THIS STUFF?
I DUNNO, LIKE...MAYBE?
imgflip.com


Seems I should write some tests…

Test Package

unittest

- in Python standard library
- class-based structure

pytest

- simple syntax
- tests are plain functions
- powerful features: fixtures, parametrisation

Set up Pytest

1. Install pytest package:
   `uv add pytest --group dev`
2. Create folder “tests” in the root directory of the package
3. Implement tests
4. Set up the tests in VSCode/Cursor (choose pytest and the tests folder)
5. Alternatively, call:
   `uv run pytest`

03

Build & Publish

Build and publish the package to a Python package registry



PyPI

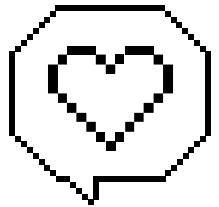
Python Package Index - a central place to store and share Python packages

- Public Python package registry
- Publish your own package
- Others can install the package easily with pip, poetry or uv



> 900.000

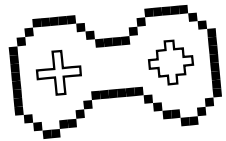
Python packages on PyPI: https://pypi.org/



Private Python Package Registry

A package registry that is not public

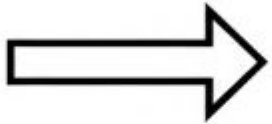
- Used inside companies or teams
- Store internal or proprietary packages
- Common options
  - Artifactory
  - Nexus
  - Self-managed GitLab



Build & Publish



Package needs to be build (like a .zip)





Build & Publish



test
`uv run pytest`

build
`uv build`

publish
`uv publish`

download
`uv add data-analytics`









Local PyPI Server

1. `uv tool install pypiserver`
2. `mkdir packages`
3. `pypi-server run -i localhost -p 8080 -a . -P . packages`

Build & Publish

1. `uv build`
2. `uv publish --publish-url http://localhost:8080`



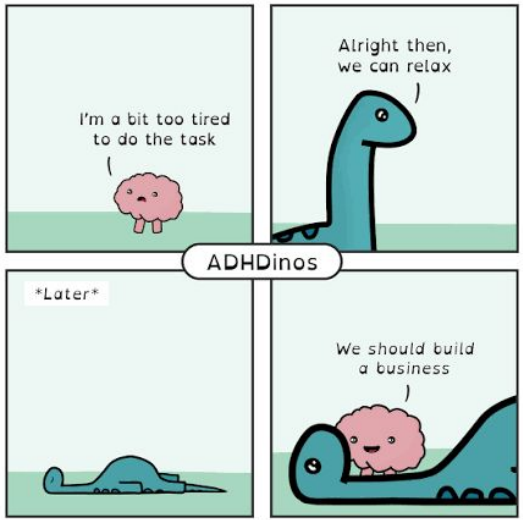
Whoa!

We did it - we published our first Python package to a Python package registry!

Search for the package:

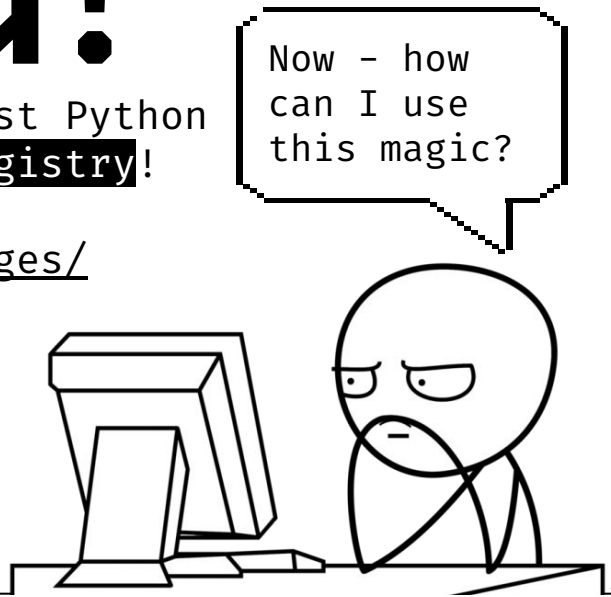
http://localhost:8080/packages/

Break!


I'm a bit too tired to do the task
Alright then, we can relax
ADHDinos
Later
We should build a business

Whoa!

We did it - we published our first Python package to a Python package registry!
Search for the package:
http://localhost:8080/packages/



04

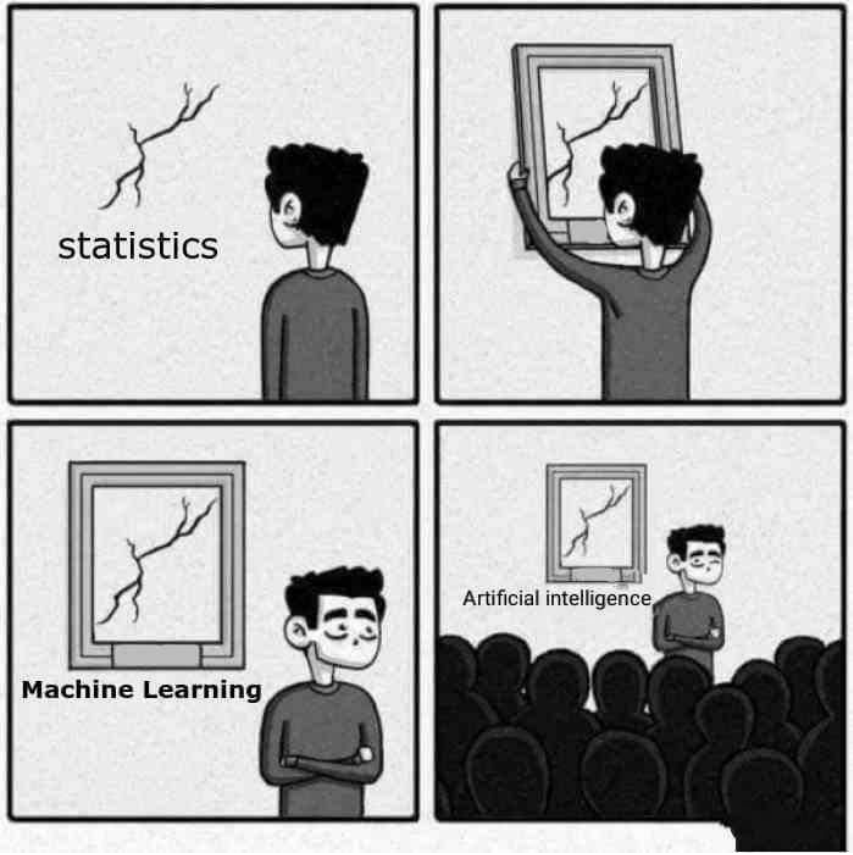
Package Integration

Install and use the package in another Python project

Using your package in another project

1. Install the package:
   ```
   uv add my-package
   ```

2. Import it in your code:
   ```
   from my_package import my_function
   result = my_function()
   print(result)
   ```


statistics
Machine Learning
Artificial intelligence

Our workshop project

Groundhog Day

Traditionally, on February 2nd, various groundhogs across North America are consulted by local public figures for an annual weather prediction:

- If the groundhog ‘sees its shadow’, it means 6 more weeks of winter
- If the groundhog doesn’t see its shadow, spring will come early

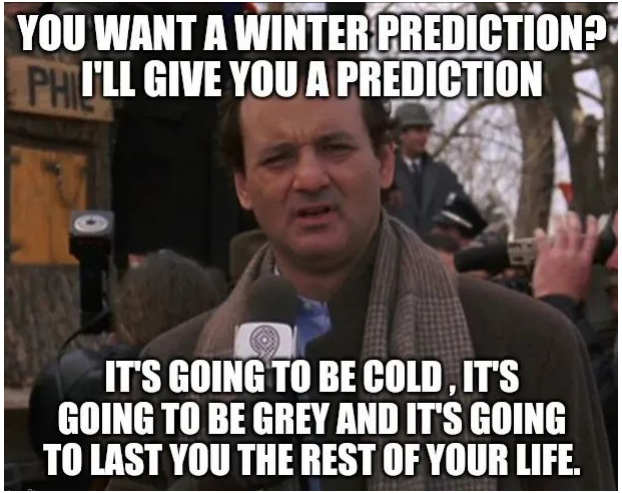


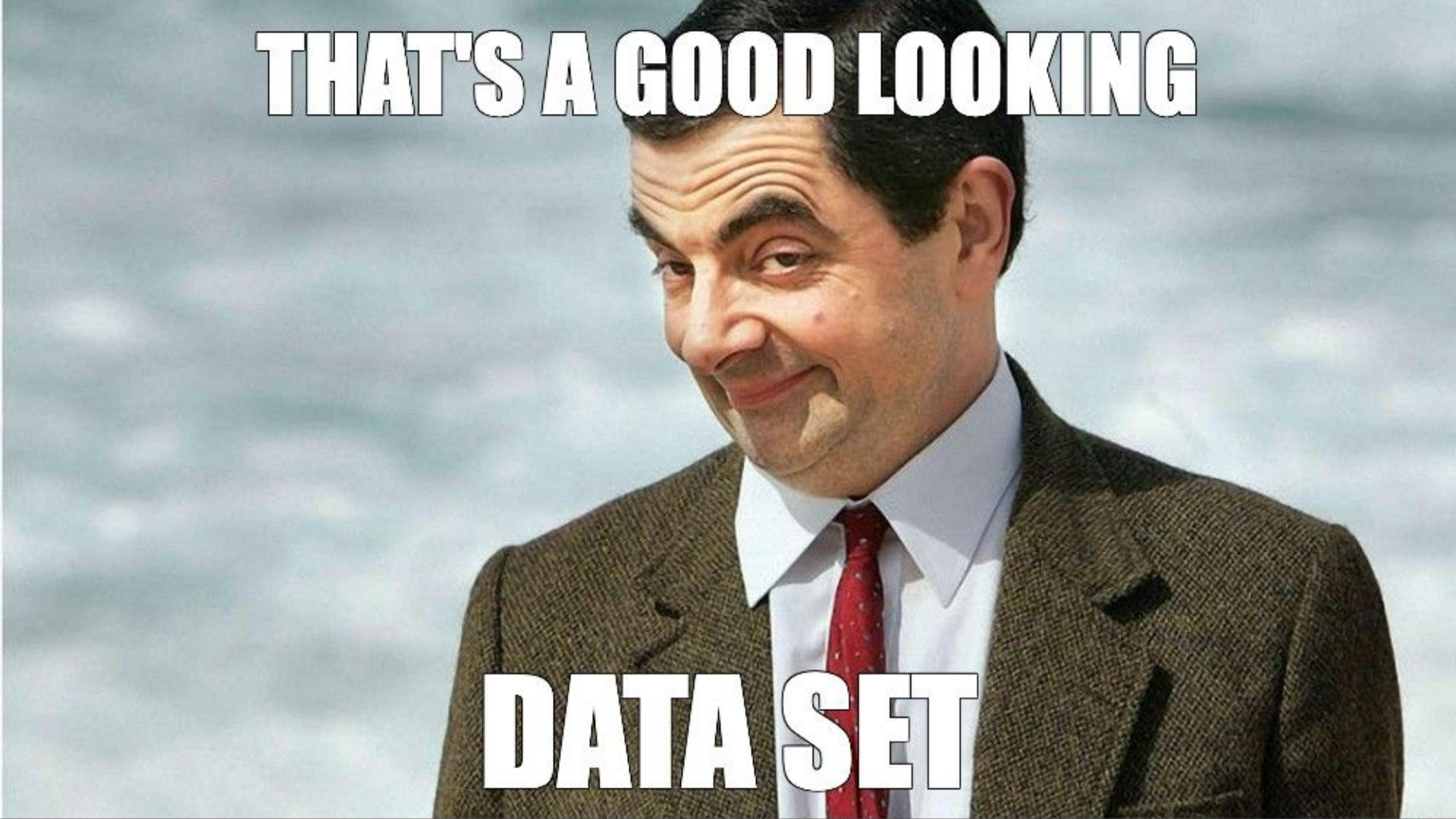


Our workshop project

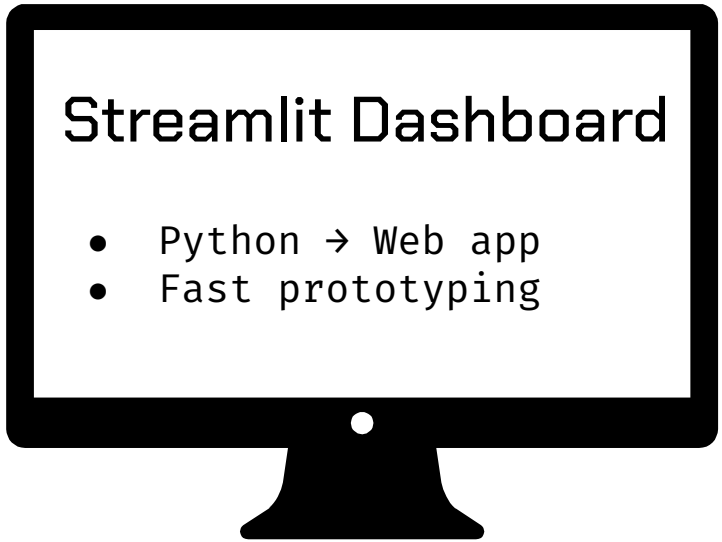
Groundhog Forecast Dashboard

- A small dashboard built with Streamlit
- Uses a public API based on predictions from Groundhog Day: https://groundhog-day.com/
- Displays infos about the groundhogs and their prediction




THAT'S A GOOD LOOKING
DATA SET

Our workshop project



Integrate Python package

Install our analytics package to get some statistics from the Groundhog Day API

Package Integration

1. Clone repo from https://github.com/kolbl/streamlit-app

2. Create virtual environment and install dependencies from pyproject.toml:
   `uv sync`

3. Remove existing package:
   `uv remove data-analytics`

4. Select Python Interpreter and install the package:
   `uv add data-analytics --index http://localhost:8080/simple/`

5. Import it in src/app.py:
   `from data_analytics import calculate_min_of_column`

Run Streamlit

1. Run streamlit app:
   `uv run streamlit run src/app.py`

2. In browser open: http://localhost:8501/

Like a machine learning model but furrier

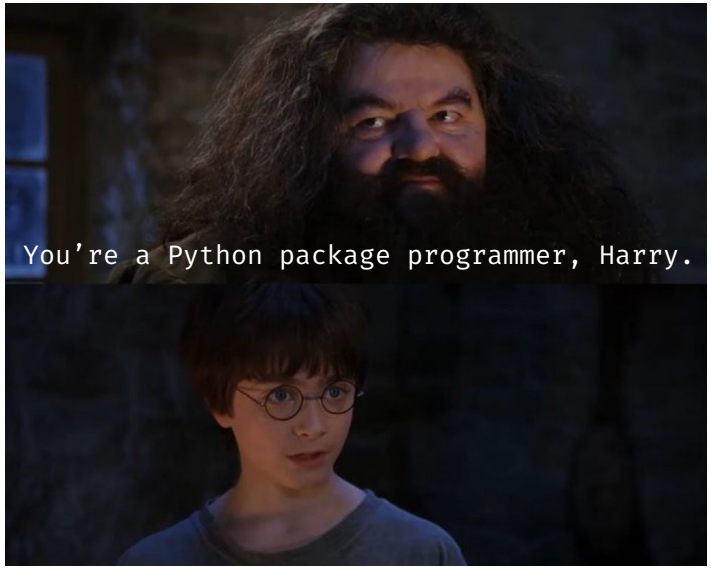


Add “span”

1. Now it’s your turn!
2. We implemented min and max together - now implement “span” (max - min) on your own!
3. Test it
4. Add new version in pyproject.toml (0.2.0)
5. Clear local dist folder
6. Build the package
7. Update the package and include the new code in Streamlit!
   uv add data-analytics==0.2.0
8. Uncomment import of the span function in Streamlit dashboard code (app.py)



Whoa!



Bonus

Dependency + Security Scans

Check your package’s dependencies and scan your code for security issues

Broccoli2Shell Vulnerability (CVE-2026-111497)

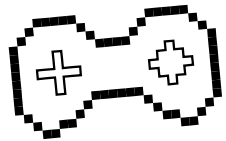
- Critical vulnerability in the Brotli decompression library
- May lead to memory corruption, crashes, or potential remote code execution
- Affects Python packages:
  - Brotli
  - brotlicffi
- Action: Upgrade to the latest patched versions



Check Dependencies with pip audit

`pip-audit` scans Python dependencies for known security vulnerabilities:

- Checks `installed packages` against public vulnerability databases
- Shows affected packages and fixed versions



Pip audit

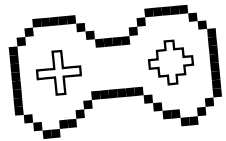
1. Install pip audit into the dev group:
   `uv add --group dev pip-audit`

2. Run the scan in the current uv env:
   `uv run pip-audit`

Bandit

bandit is a static application security testing (SAST) tool for Python

- Scans source code for common security issues
  - Hardcoded passwords/secrets
  - Use of eval() and unsafe deserialization
  - Weak cryptography
  - Command injection risks
- Integrates easily into CI/CD pipelines



Bandit

1. Install bandit into the dev group:
   `uv add --group dev bandit`

2. Run the scan for your code files
   `uv run bandit -r src/data_analytics -f txt`

What we did today

created package structure

added dependencies

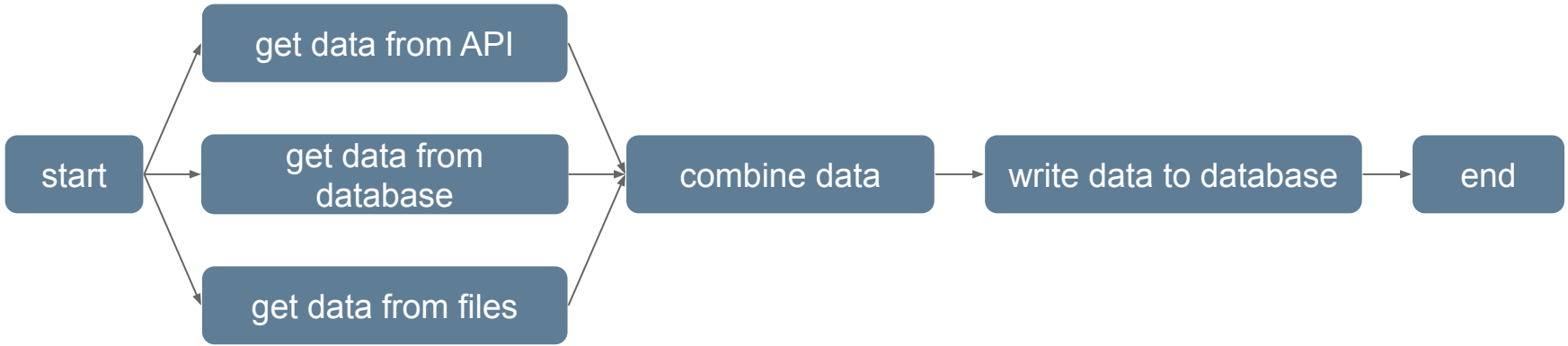
wrote package code

built the package

published the package

integrated the package

ETL Pipeline



ETL Pipeline

start → get data from groundhog day API → calculate statistics → end



+



=



Thanks!

Do you have any questions?

If you want to connect on LinkedIn: